

Chapter 04. 분류

Week 3

1 분류(Classification)의 개요

- 지도 학습

Lable(정답)이 있는 데이터로 학습하는 머신러닝 방식

- 분류(Classification)

- 지도 학습의 대표 유형

- 학습 데이터 - 데이터의 feature와 label값(결정 값, 클래스 값)

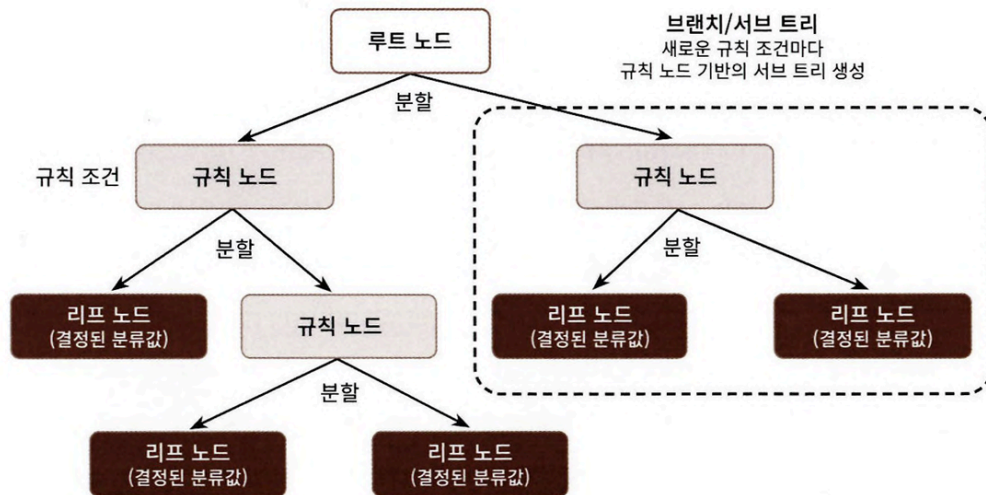
→ 새로운 데이터에 대한 미지의 레이블 값 예측

⇒ '기존 데이터가 어떤 레이블?' 패턴을 알고리즘으로 인지

2 결정 트리(Decision Tree)

결정 트리

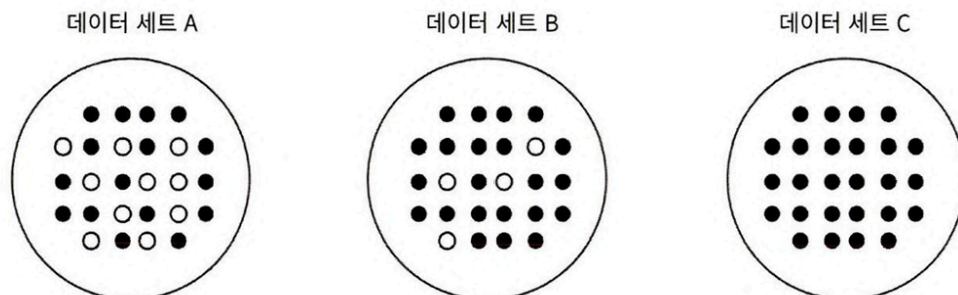
- 데이터에 있는 규칙을 학습을 통해 자동으로 찾는 트리(Tree) 기반 분류 규칙
- if/else 기반
- 결정 트리의 구조



- 규칙 노드 (Decision Node): 규칙 조건이 되는 노드
 - 리프 노드 (Leaf Node): 결정된 클래스 값
- 규칙이 많다 = 분류 결정 방식 복잡하다
 - 과적합 가능성 높음
 - ⇒ 트리의 깊이 깊어질수록 결정 트리 예측 성능 저하 가능성 높음

트리의 분할(Split)

- 최대한 균일한 데이터 세트 구성하도록 분할
 - 최대한 많은 데이터 세트가 해당 분류에 속하도록 결정 노드 규칙정해야 함
 - 균일도 $C > B > A$ (C는 모두 검은 공)



- 균일도는 데이터 구분에 필요한 정보의 양에 영향 미침

- C는 어떤 데이터를 뽑아도 검은 공 예측 가능
- A는 같은 조건 상 데이터 판단에 더 많은 정보 필요
 - 상대적으로 혼잡도↑, 균일도↓

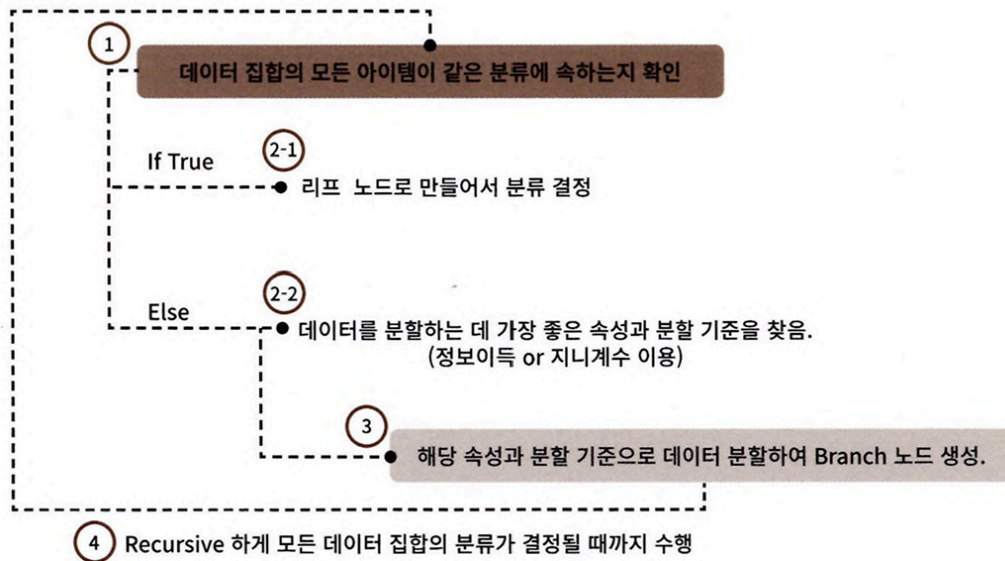
결정노드

- 정보 균일도 높은 데이터 세트 먼저 선택하도록 규칙 조건
→ 서브 데이터 세트 생성
- 반복
- e.g) 여러 형태와 색깔의 레고 블록, 노란색 블록은 모두 동그라미
 - 첫 조건 if 색깔 == '노란색'

정보 이득(Information Gain) 지수와 지니 계수

정보 균일도 측정 대표적 방법

- 엔트로피 개념 기반
 - 엔트로피: 주어진 데이터 집합의 혼잡도
 - 서로 다른 값 섞여 있으면 엔트로피 높음, 같은 값 섞이면 낮음
- 정보 이득 지수
 - (1- 엔트로피 지수)
 - 결정 트리는 정보 이득 높은 (= 엔트로피 낮은 = 같은 값 섞인 = 균일도 높은) 속성을 기준으로 분할
- 지니 계수
 - 경제학의 불평등 지수
 - 0이 가장 평등, 1로 갈수록 불평등
 - 지니 계수 낮을수록 데이터 균일도 높다고 해석
 - 지니 계수 낮은 속성 기준 분할



결정 트리 모델의 특징

결정 트리 모델의 장점

- '균일도' 기반
 - 알고리즘이 쉽고 직관적
- 룰 명확, 시각화 표현 가능
- 균일도만 신경 쓰면 됨
 - 각 feature의 스케일링과 정규화 등의 전처리 작업 필요 없음

결정 트리 모델의 단점

- 과적합으로 정확도 떨어짐
- feature 많고 균일도 다양
 - 트리 깊이 커지고 복잡해짐

⇒ 트리 크기를 사전에 제한하는 튜닝 필요

결정 트리 파라미터

사이킷런 제공 클래스

- DecisionTreeClassifier
 - 분류 위한 클래스
- DecisionTreeRegressor
 - 회귀 위한 클래스
- 사이킷런 결정 트리 구현은 CART(Classification And Regression Trees) 알고리즘 기반

파라미터

파라미터 명	설명
min_samples_split	- 노드 분할 위한 최소 샘플 데이터 수 - 과적합 제어에 사용 - 디폴트는 2 - 작게 설정할수록 분할 노드 많아지면서 과적합 가능성 높아짐
min_samples_leaf	- 분할될 경우 왼쪽과 오른쪽 브랜치 노드에서 가져야 할 최소한 샘플 데이터 수 - 큰 값으로 설정될수록 노드 분할 상대적으로 덜 수행함 ($\because$ 분할될 경우 왼쪽과 오른쪽의 브랜치 노드에서 가져가야 할 최소한의 샘플 데이터 수 조건 만족시키기 어려움) - min_samples_split과 유사하게 과적합 제어 용도, but 비대칭적 데이터는 특정 클래스의 데이터가 극도로 작을 수 있으므로 이 경우는 작게 설정 필요
max_features	- 최적의 분할 위해 고려할 최대 feature 개수 - 디폴트는 None, 데이터 세트의 모든 feature를 사용해 분할 수행 - int 형으로 지정하면 대상 feature의 개수 - float 형으로 지정하면 전체 feature 중 대상 feature의 퍼센트 - 'sqrt'는 전체 feature 중 $\sqrt{\text{전체 feature}}$ 개수 $\Rightarrow$ (전체 feature 개수)$^{1/2}$ 만큼 선정 - 'auto'로 지정하면 sqrt와 동일 - 'log'는 전체 feature 중 $\log_2(\text{전체 feature 개수})$ 선정 - 'None'은 전체 feature 선정

파라미터 명	설명
max_depth	- 트리의 최대 깊이 규정 - 디폴트는 None - None으로 설정하면 완벽하게 클래스 결정 값이 될 때까지 깊이를 계속 키우며 분할 or 노드가 가지는 데이터 개수가 min_samples_Split보다 작아질 때까지 계속 깊이 증가 - 깊이 깊어지면 min_sample_split 설정대로 최대 분할하여 과적합할 수 있으므로 적절 값으로 제어 필요
max_leaf_nodes	- 말단 노드(Leaf)의 최대 개수

결정 트리 모델 시각화

- Graphviz 패키지 사용
 - 원그래프 기반의 dot 파일로 쉽게 시각화 가능 패키지
 - 사이킷런에서 export_graphviz() API 제공

Graphviz 설치

1. 윈도우 버전 Graphviz 내려 받은 뒤 설치
2. Graphviz의 파이썬 래퍼 모듈을 명령어 통해 설치
3. 주피터 노트북 재기동

Graphviz 이용해 붓꽃 데이터 세트 시각화

```

제안된 코드에 라이선스가 적용될 수 있습니다.
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

#DecisionTree Classifier 생성
dt_clf = DecisionTreeClassifier(random_state=156)

iris_data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target, test_size=0.2, random_state=11)

dt_clf.fit(X_train, y_train)

```

```

DecisionTreeClassifier
DecisionTreeClassifier(random_state=156)

```

```

from sklearn.tree import export_graphviz

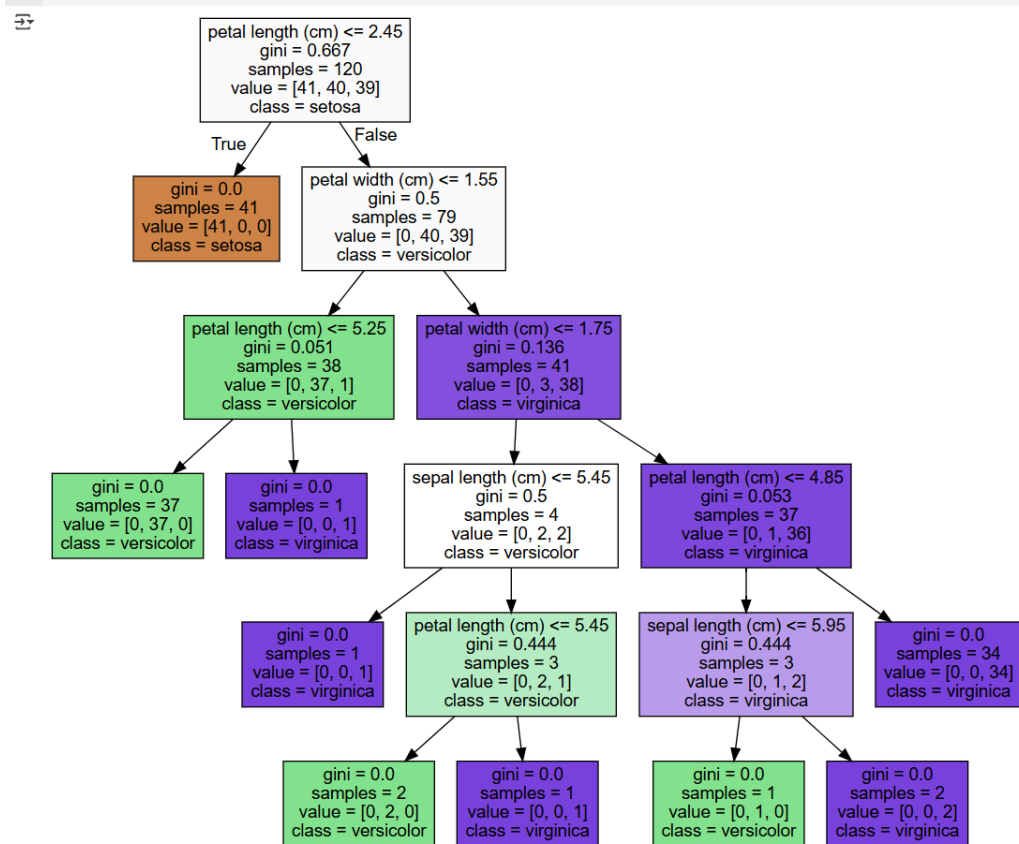
export_graphviz(dt_clf, out_file = "tree.dot", class_names = iris_data.target_names, #
                feature_names = iris_data.feature_names, impurity=True, filled=True)

```

- `export_graphviz()`
 - Graphviz가 읽어 들어서 시각화할 수 있는 출력 파일 생성
 - 학습 완료된 estimator, output 파일 명, 결정 클래스 명칭, feature 명칭 입력하면 됨
- `tree.dot`을 Graphviz 파이썬 모듈을 호출해 시각화

```
import graphviz

with open("tree.dot") as f:
    dot_graph = f.read()
graphviz.Source(dot_graph)
```



- 각 트리의 브랜치 노드와 말단 리프 노드가 어떻게 구성되는지 시각화
- 리프 노드(Leaf Node)
 - 더 이상 자식 노드가 없는 노드
 - 최종 클래스(레이블) 값이 결정되는 노드
 - 리프 노드 되려면 오직 하나의 클래스 값으로 최종 데이터 구성 or 리프 노드 될 수 있는 하이퍼 파라미터 조건 충족
- 브랜치 노드(Branch Node)
 - 자식 노드 만들기 위한 분할 규칙 조건 가짐
- (cm)≤
 - feature 조건, 자식 노드 만들기 위한 규칙 조건
 - 없으면 리프 노드
- gini
 - 다음의 value=[]로 주어진 데이터 분포에서의 지니 계수

- samples
 - 현 규칙에 해당하는 데이터 건수
- value = []
 - 클래스 값 기반의 데이터 건수
 - e.g.) value = [41, 40, 39]
 - Setosia 41개, Versicolor 40개, Virginica 39개
- 각 노드의 색깔
 - 붓꽃 데이터의 레이블 값
 - 색깔 짙을수록 지니 계수 낮음, 해당 레이블 속한 샘플 데이터 많음
- 노드가 분할될 경우 자식 노드들이 모두 샘플 데이터 건수가 4 인상을 만족할 수 있는지를 확인한 후 분할 수행
 - 결정 트리는 해당조건 반영하여 트리 구조 변경
- 결정 트리는 균일도에 기반해 어떠한 속성을 규칙 조건으로 선택하느냐가 중요한 요건
 - 중요한 몇 개의 feature가 명확한 규칙 트리를 만드는 데 크게 기여
 - 모델을 더 간결하고 이상치(Outlier)에 강한 모델을 만들 수 있기 때문

feature_importances_

- feature가 트리 분할 시 정보 이득이나 지니 계수를 얼마나 효율적으로 잘 개성했는지 정규화된 값으로 표현
- 일반적으로 값 높을수록 해당 feature의 중요도 높음 의미(예외 존재)
- DecisionTreeClassifier 객체의 feature_importances_속성
 - 사이킷런이 제공하는 결정 트리 알고리즘이 학습 통해 규칙 정하는 데 있어 feature의 중요 역할 지표
- ndarray 형태로 값 반환
- feature 순서대로 값 할당
- 붓꽃 데이터 세트 feature 별 결정 트리 알고리즘 중요도 추출

- 여러 feature 중 petal_length feature 중요도 가장 높음

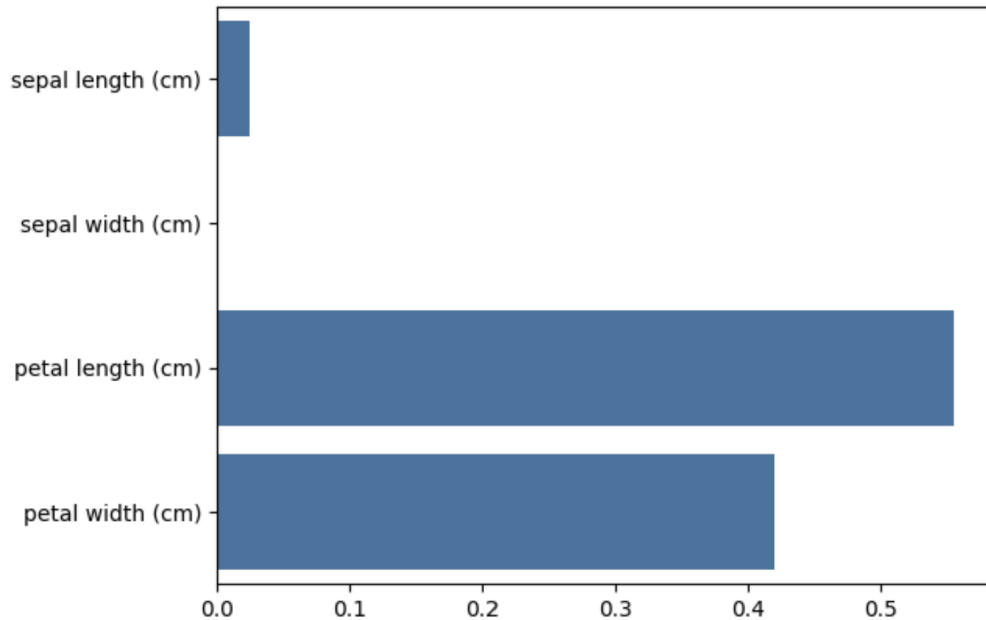
```
import seaborn as sns
import numpy as np
%matplotlib inline

print("Feature importances:\n{0}".format(np.round(dt_clf.feature_importances_, 3)))

for name, value in zip(iris_data.feature_names, dt_clf.feature_importances_):
    print('{0} : {1:.3f}'.format(name, value))

sns.barplot(x=dt_clf.feature_importances_, y=iris_data.feature_names)
```

```
Feature importances:
[0.025 0.    0.555 0.42 ]
sepal length (cm) : 0.025
sepal width (cm) : 0.000
petal length (cm) : 0.555
petal width (cm) : 0.420
<Axes: >
```



결정 트리 과적합(Overfitting)

분류 위한 데이터 세트 생성

- 2개의 feature가 3가지 유형의 클래스 값을 가지는 데이터 세트 생성, 그래프 형태로 시각화
- make_classification() 호출 시 반환되는 객체는 feature 데이터 세트와 클래스 레이블 데이터 세트

```

from os import XATTR_SIZE_MAX
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt
%matplotlib inline

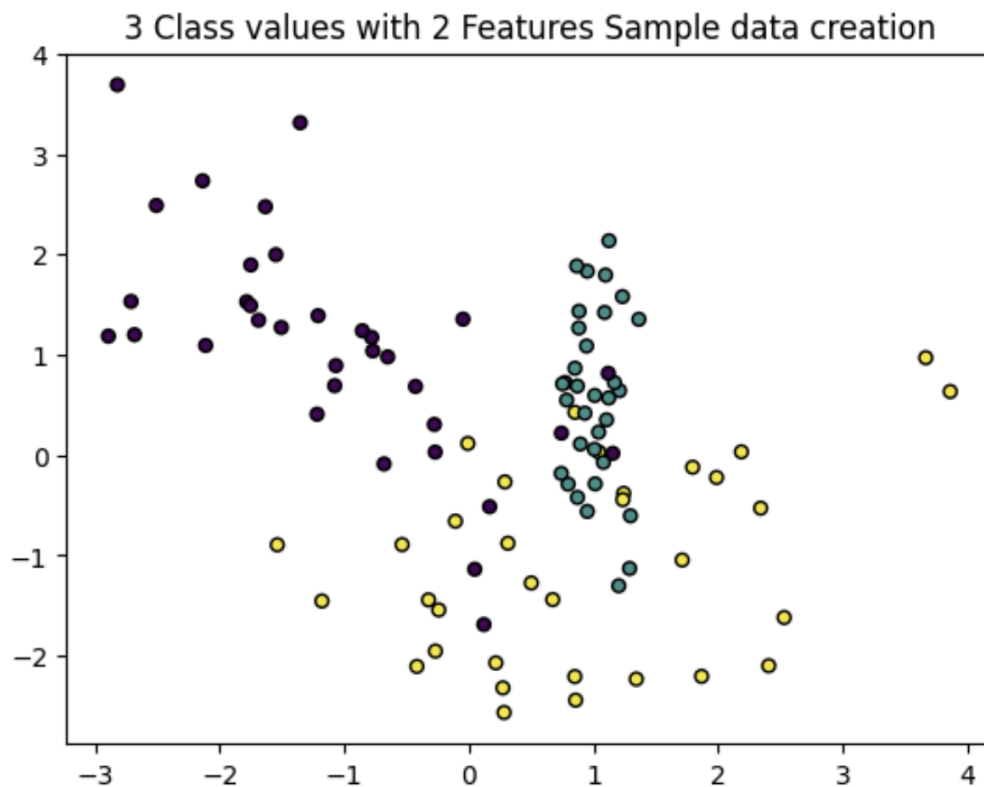
plt.title('3 Class values with 2 Features Sample data creation')

X_features, y_labels = make_classification(n_features=2, n_redundant=0, n_informative=2,
                                          n_classes=3, n_clusters_per_class=1, random_state=0)

plt.scatter(X_features[:, 0], X_features[:, 1], marker='o', c=y_labels, s=25, edgecolor='k')

```

<matplotlib.collections.PathCollection at 0x7d0be1d0fd90>



- 각 feature가 X, Y축으로 나열된 2차원 그래프
- 3개의 클래스 값 구분은 색깔

결정 트리 실습 - 사용자 행동 인식 데이터 세트

결정 트리 이용 UCI 머신러닝 레포지토리에서 제공하는 사용자 행동 인식 데이터 세트에 대한 예측 분류

- feature는 모두 561개, 공백 분리
- features.txt 파일은 feature 인덱스와 feature명 가짐

- DataFrame으로 로딩하여 feature 명칭 간략 확인

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

# features.txt 파일에는 피쳐 이름 index와 피쳐명이 공백으로 분리되어 있음. 이를 DataFrame으로 로드.
feature_name_df = pd.read_csv('features.txt', sep='\\s+',
                             header=None, names=['column_index', 'column_name'])

# 피쳐명 index를 제거하고, 피쳐명만 리스트 객체로 생성한 뒤 샘플로 10개만 추출
feature_name = feature_name_df.iloc[:, 1].values.tolist()
print('전체 피쳐명에서 10개만 추출:', feature_name[:10])

전체 피쳐명에서 10개만 추출: ['tBodyAcc-mean()-X', 'tBodyAcc-mean()-Y', 'tBodyAcc-mean()-Z', 'tBodyAcc-std()-X', 'tBodyAcc-std()-Y', 'tBodyAcc-std()-Z', 'tBodyAcc-mad()-X', 'tBodyAcc-mad-
```

- 인체의 움직임과 관련된 속성의 평균/표준편차가 X,Y,Z축 값으로 되어 있음
- feature 명을 가지는 DataFrame을 이용해 데이터 파일을 데이터 세트 DataFrame에 로딩하기 전 유의 사항
 - feature명 가지는 feature.txt 파일은 중복된 feature명 가짐
 - 중복된 feature명들을 이용해 데이터 파일을 데이터 세트 DataFrame에 로드 하면 오류 발생
 - 중복된 feature명에 대해서는 원본 feature명에 _1 혹은 _2를 추가로 부여해 변경한 뒤 데이터를 DataFrame에 로드
- 중복된 feature명 얼마나 있는지 확인하기

```
feature_dup_df = feature_name_df.groupby('column_name').count()
print(feature_dup_df[feature_dup_df['column_index'] > 1].count())
feature_dup_df[feature_dup_df['column_index'] > 1].head()
```

```
column_index    42
dtype: int64
```

column_index	
column_index	column_name
3	fBodyAcc-bandsEnergy()-1,16
3	fBodyAcc-bandsEnergy()-1,24
3	fBodyAcc-bandsEnergy()-1,8
3	fBodyAcc-bandsEnergy()-17,24
3	fBodyAcc-bandsEnergy()-17,32

- 총 42개 feature명 중복

- 원본 feature명에 _1 / _2 부여해 새로운 feature명 가지는 DataFrame 반환하는 함수인 get_new_feature_name_df() 생성

```
def get_new_feature_name_df(old_feature_name_df):
    feature_dup_df = pd.DataFrame(data=old_feature_name_df.groupby('column_name').cumcount(),
                                   columns=['dup_cnt'])
    feature_dup_df = feature_dup_df.reset_index()
    new_feature_name_df = pd.merge(old_feature_name_df.reset_index(), feature_dup_df, how='outer')
    new_feature_name_df['column_name'] = new_feature_name_df[['column_name', 'dup_cnt']].apply(lambda x : x[0]+'_'+str(x[1])
                                                                                               if x[1] >0 else x[0] , axis=1)

    new_feature_name_df = new_feature_name_df.drop(['index'], axis=1)
    return new_feature_name_df
```

- train 디렉터리에 있는 학습용 feature / label 데이터 세트, test 디렉터리에 있는 테스트용 feature / label 데이터 파일 각각 DataFrame에 업로드

```
import pandas as pd

def get_human_dataset( ):

    # 각 데이터 파일들은 공백으로 분리되어 있으므로 read_csv에서 공백 문자를 sep으로 할당.
    feature_name_df = pd.read_csv('features.txt',sep='Ws+',
                                   header=None,names=['column_index', 'column_name'])

    # 중복된 피처명을 수정하는 get_new_feature_name_df()를 이용, 신규 피처명 DataFrame생성.
    new_feature_name_df = get_new_feature_name_df(feature_name_df)

    # DataFrame에 피처명을 컬럼으로 부여하기 위해 리스트 객체로 다시 변환
    feature_name = new_feature_name_df.iloc[:, 1].values.tolist()

    # 학습 피쳐 데이터 셋과 테스트 피쳐 데이터를 DataFrame으로 로딩. 컬럼명은 feature_name 적용
    X_train = pd.read_csv('X_train.txt',sep='Ws+', names=feature_name )
    X_test = pd.read_csv('X_test.txt',sep='Ws+', names=feature_name)

    # 학습 레이블과 테스트 레이블 데이터를 DataFrame으로 로딩하고 컬럼명은 action으로 부여
    y_train = pd.read_csv('y_train.txt',sep='Ws+',header=None,names=['action'])
    y_test = pd.read_csv('y_test.txt',sep='Ws+',header=None,names=['action'])

    # 로드된 학습/테스트용 DataFrame을 모두 반환
    return X_train, X_test, y_train, y_test

X_train, X_test, y_train, y_test = get_human_dataset()
```

- 각 데이터 파일은 공백 분리, read_csv()의 sep 인자로 공백 문자 입력
 - label 칼럼은 action으로 명명
 - 함수명은 get_human_dataset()
- 사이킷런 DecisionTreeClassifier 이용해 동작 예측 분류 수행

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 예제 반복 시 마다 동일한 예측 결과 도출을 위해 random_state 설정
dt_clf = DecisionTreeClassifier(random_state=156)
dt_clf.fit(X_train, y_train)
pred = dt_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred)
print('결정 트리 예측 정확도: {0:.4f}'.format(accuracy))

# DecisionTreeClassifier의 하이퍼 파라미터 추출
print('DecisionTreeClassifier 기본 하이퍼 파라미터:\n', dt_clf.get_params())

```

결정 트리 예측 정확도: 0.8548
DecisionTreeClassifier 기본 하이퍼 파라미터:
{'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': None, 'max_leaf_nodes': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_s

- DecisionTreeClassifier의 하이퍼 파라미터는 모두 디폴트값 설정
 - 하이퍼 파라미터 값 모두 추출
 - 약 85.48%의 정확도
- 결정 트리의 Tree Depth가 예측 정확도에 주는 영향
 - 결정 트리는 분류를 위해 leaf node(클래스 결정 노드)가 될 수 있는 적합한 수준이 될 때까지 지속해서 트리 분할 수행, 깊이 깊어짐
 - GridSearchCV 이용, 사이킷런 결정 트리의 깊이 조절할 수 있는 하이퍼 파라미터인 max_depth 값 변화, 예측 성능 확인

```

from sklearn.model_selection import GridSearchCV

params = {
    'max_depth' : [ 6, 8, 10, 12, 16, 20, 24],
    'min_samples_split' : [16]
}

grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5, verbose=1)
grid_cv.fit(X_train, y_train)
print('GridSearchCV 최고 평균 정확도 수치:{0:.4f}'.format(grid_cv.best_score_))
print('GridSearchCV 최적 하이퍼 파라미터:', grid_cv.best_params_)

```

Fitting 5 folds for each of 7 candidates, totalling 35 fits
GridSearchCV 최고 평균 정확도 수치:0.8549
GridSearchCV 최적 하이퍼 파라미터: {'max_depth': 8, 'min_samples_split': 16}

- min_samples_split은 16 고정, max_depth를 6, 8, 10, 12, 16, 20, 24로 늘리면서 예측 성능 측정
- 교차 검증은 5개 세트
- max_depth가 8일 때 5개 fold 세트의 최고 평균 정확도 결과가 약 85.49%

- 5개의 CV 세트에서 `max_depth`값에 따른 예측 성능 변화

```
# GridSearchCV객체의 cv_results_ 속성을 DataFrame으로 생성.
cv_results_df = pd.DataFrame(grid_cv.cv_results_)

# max_depth 파라미터 값과 그때의 테스트(Evaluation)셋, 학습 데이터 셋의 정확도 수치 추출
cv_results_df[['param_max_depth', 'mean_test_score']]
```

	param_max_depth	mean_test_score
0	6	0.850791
1	8	0.851069
2	10	0.851209
3	12	0.844271
4	16	0.850255
5	20	0.850120
6	24	0.849168

- `cv_results_` 속성으로 확인
 - `mean_test_score`는 `max_depth` 8일 때 0.854로 정확도 정점
-
- `min_samples_split` 16 고정, `max_depth` 변화에 따른 값 측정
 - `max_depth` 8일 때 87.17%로 가장 높은 정확도
 - `max_depth` 8 넘어가면서 정확도 감소
 - 결정 트리는 깊어질수록 과적합 영향력 커짐

```
max_depths = [ 6, 8 ,10, 12, 16 ,20, 24]
# max_depth 값을 변화 시키면서 그때마다 학습과 테스트 셋에서의 예측 성능 측정
for depth in max_depths:
    dt_clf = DecisionTreeClassifier(max_depth=depth, min_samples_split=16, random_state=156)
    dt_clf.fit(X_train , y_train)
    pred = dt_clf.predict(X_test)
    accuracy = accuracy_score(y_test , pred)
    print('max_depth = {0} 정확도: {1:.4f}'.format(depth , accuracy))
```

```
max_depth = 6 정확도: 0.8551
max_depth = 8 정확도: 0.8717
max_depth = 10 정확도: 0.8599
max_depth = 12 정확도: 0.8571
max_depth = 16 정확도: 0.8599
max_depth = 20 정확도: 0.8565
max_depth = 24 정확도: 0.8565
```

- `max_depth`와 `min_samples_split` 함께 변경

```

params = {
    'max_depth' : [ 8 , 12, 16 ,20],
    'min_samples_split' : [16, 24],
}

grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5, verbose=1 )
grid_cv.fit(X_train , y_train)
print('GridSearchCV 최고 평균 정확도 수치: {0:.4f}'.format(grid_cv.best_score_))
print('GridSearchCV 최적 하이퍼 파라미터:', grid_cv.best_params_)

Fitting 5 folds for each of 8 candidates, totalling 40 fits
GridSearchCV 최고 평균 정확도 수치: 0.8549
GridSearchCV 최적 하이퍼 파라미터: {'max_depth': 8, 'min_samples_split': 16}

```

- max_depth 8, min_samples_split 16일 때 최고 정확도 85.49%
- GridSearchCV 객체인 grid_cv의 속성인 best_estimator_는 최적 하이퍼 파라미터인 max_depth 8, min_samples_split 16으로 학습 완료된 Estimator 객체

```

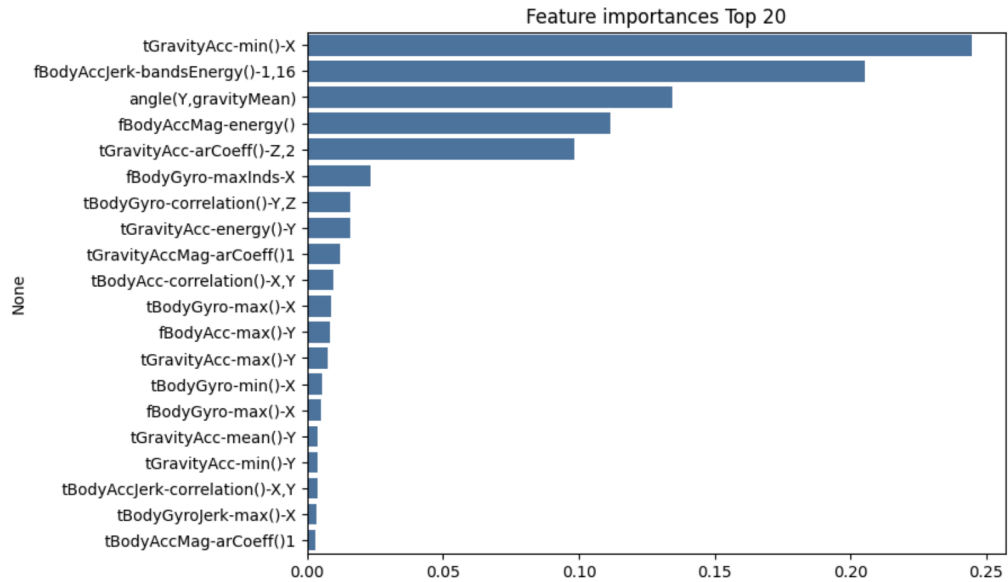
best_df_clf = grid_cv.best_estimator_
pred1 = best_df_clf.predict(X_test)
accuracy = accuracy_score(y_test , pred1)
print('결정 트리 예측 정확도:{0:.4f}'.format(accuracy))

```

결정 트리 예측 정확도:0.8673

- max_depth 8, min_samples_split 16일 때 테스트 데이터 세트의 예측 정확도 87.17%
- 결정 트리에서 각 feature의 중요도를 feature_importances_ 속성으로 표현


```
import seaborn as sns
ftr_importances_values = best_df_clf.feature_importances_
# Top 중요도로 정렬을 쉽게 하고, 시본(Seaborn)의 막대 그래프로 쉽게 표현하기 위해 Series변환
ftr_importances = pd.Series(ftr_importances_values, index=X_train.columns)
# 중요도값 순으로 Series를 정렬
ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]
plt.figure(figsize=(8,6))
plt.title('Feature importances Top 20')
sns.barplot(x=ftr_top20, y=ftr_top20.index)
plt.show()
```



3 앙상블 학습

앙상블 학습 개요

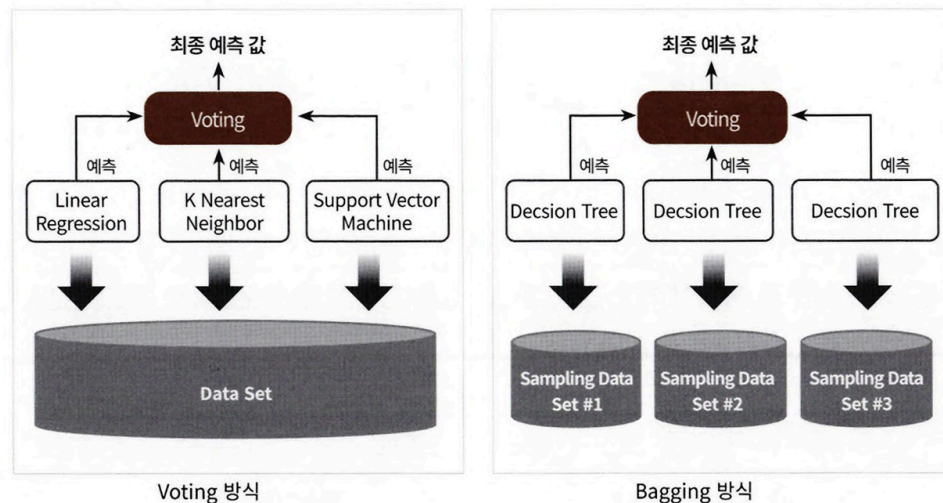
앙상블 학습(Ensemble Learning)을 통한 분류

- 여러 개의 분류기(Classifier)를 생성하고 그 예측을 결합함으로써 더 정확한 최종 예측 도출하는 기법
- 목표
 - 다양한 분류기의 예측 결과를 결합함으로써 단일 분류기보다 신뢰성이 높은 예측값 얻는 것
- 비정형 데이터(이미지, 영상, 음성 등)의 데이터 분류는 딥러닝이 뛰어난 성능
- but 대부분의 정형 데이터 분류는 앙상블이 뛰어난 성능
- 앙상블 알고리즘의 대표
 - 랜덤 포레스트

- 그래디언트 부스팅 알고리즘

앙상블 학습의 유형

- 보팅(Voting), 배깅(Bagging), 부스팅(Boosting)
- 보팅, 배깅

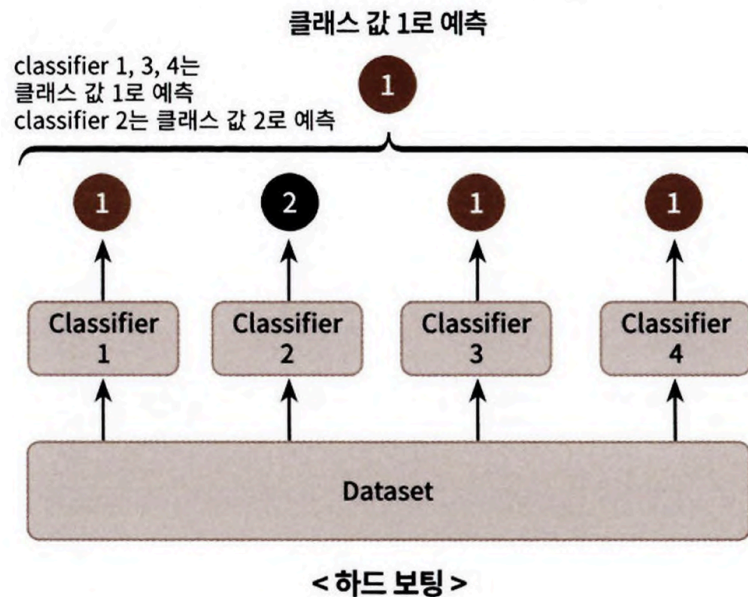


- 여러 개의 분류기가 투표를 통해 최종 예측 결과 정하는 방법
- 보팅
 - 일반적으로 서로 다른 알고리즘을 가진 분류기를 결합
 - 선형 회귀, K 최근접 이웃, SVM 3개의 ML 알고리즘이 같은 데이터 세트에 대해 학습하고 예측한 결과를 가지고 보팅 통해 최종 예측 결과 선정
- 배깅
 - 각각의 분류기가 모두 같은 유형의 알고리즘 기반이지만, 데이터 샘플링 서로 다르게 가져가며 학습 수행
 - 랜덤 포레스트 알고리즘
 - 단일 ML 알고리즘(결정 트리)으로 여러 분류기가 학습으로 개별 예측
 - 학습하는 데이터 세트가 보팅 방식과는 다름
 - 개별 분류기에 할당된 학습 데이터는 원본 학습 데이터를 샘플링하여 추출
 - 개별 Classifier에게 데이터를 샘플링하여 추출하는 방식
 - ⇒ 부트스트래핑(Bootstrapping) 분할 방식

보팅 유형 - 하드 보팅과 소프트 보팅

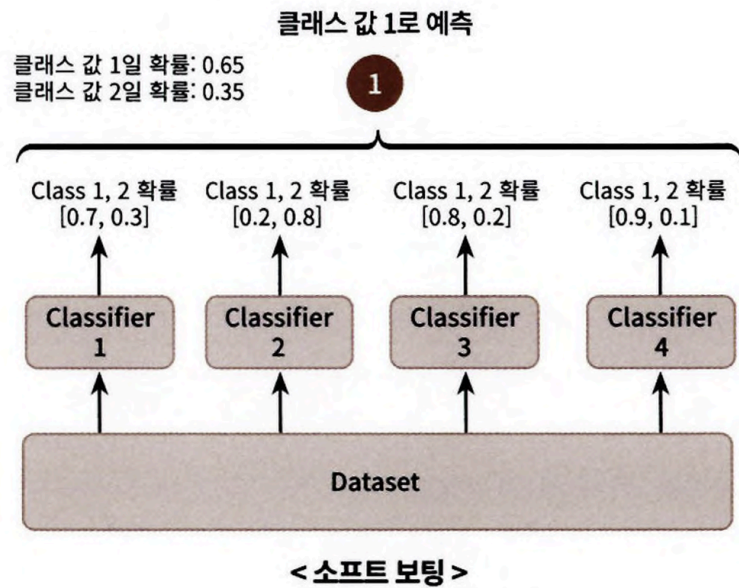
- 하드 보팅 - 다수결 원칙과 비슷
 - 예측한 결과값들 중 다수의 분류기가 결정한 예측값을 최종 보팅 결과값으로 선정

Hard Voting은 다수의 classifier 간 다수결로 최종 class 결정



- Classifier 1번, 2번, 3번, 4번인 4개로 구성된 보팅 앙상블 기법
 - ⇒ 다수결 원칙에 따라 최종 예측 레이블 값 1
 - 일반적으로 보팅 방법에 적용됨
 - 분류기 1, 3, 4 → 1 / 분류기 2 → 2 예측
- 소프트 보팅
 - 분류기들의 레이블 값 결정 확률을 모두 더하고 이를 평균하여 확률 가장 높은 레이블 값을 최종 보팅 결과값으로 선정

Soft Voting은 다수의 classifier들의 class 확률을 평균하여 결정



- Class 1 확률 $(0.7 + 0.2 + 0.8 + 0.9) / 4 = 0.65$
- Class 2 확률 $(0.3 + 0.8 + 0.2 + 0.1) / 4 = 0.35$
- $0.65 > 0.35$ 이므로 레이블 값 1로 최종 보팅
- 일반적으로 하드 보팅보다 예측 성능 좋음

보팅 분류기(Voting Classifier)

- 사이킷런 - 보팅 방식 앙상블 구현한 VotingClassifier 클래스 제공

위스콘신 유방암 데이터 세트 예측 분석

- load_breast_cancer() 함수로 자체에서 위스콘신 유방암 데이터 세트 생성 가능

```

import pandas as pd

from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer = load_breast_cancer()

data_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
data_df.head(3)
  
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension
0	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.6	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890
1	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.8	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902
2	19.69	21.25	130.0	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.5	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758

3 rows x 30 columns

- 로지스틱 회귀와 KNN 기반

- 로지스틱 회귀와 KNN 기반의 소프트 보팅 방식 보팅 분류기 생성

```
# 개별 모델은 로지스틱 회귀와 KNN 임.
lr_clf = LogisticRegression(solver='liblinear')
knn_clf = KNeighborsClassifier(n_neighbors=8)

# 개별 모델을 소프트 보팅 기반의 앙상블 모델로 구현한 분류기
vo_clf = VotingClassifier( estimators=[('LR',lr_clf),('KNN',knn_clf)] , voting='soft' )

X_train, X_test, y_train, y_test = train_test_split(cancer.data, cancer.target,
                                                    test_size=0.2 , random_state= 156)

# VotingClassifier 학습/예측/평가.
vo_clf.fit(X_train , y_train)
pred = vo_clf.predict(X_test)
print('Voting 분류기 정확도: {0:.4f}'.format(accuracy_score(y_test , pred)))

# 개별 모델의 학습/예측/평가.
classifiers = [lr_clf, knn_clf]
for classifier in classifiers:
    classifier.fit(X_train , y_train)
    pred = classifier.predict(X_test)
    class_name= classifier.__class__.__name__
    print('{0} 정확도: {1:.4f}'.format(class_name, accuracy_score(y_test , pred)))

Voting 분류기 정확도: 0.9561
LogisticRegression 정확도: 0.9474
KNeighborsClassifier 정확도: 0.9386
```

- 정확도 조금 높긴 하나, 보팅으로 여러 개의 기반 분류기를 결합한다고 무조건 기반 분류기보다 예측 성능 향상되지는 않음
 - 데이터 특성, 분포 등에 따라 오히려 가장 좋은 분류기 성능이 보팅보다 나올 수 있음
- VotingClassifier 클래스는 주요 생성 인자로 estimators, voting 값 입력받음
- estimators
 - 리스트 값으로 보팅에 사용될 여러 개의 Classifier 객체들을 튜플 형식으로 입력 받음
- Voting
 - 'hard'시 하드 보팅, 'soft'시 소프트 보팅
 - 기본은 hard

- 편향-분산 트레이드오프는 ML 모델이 극복해야 할 과제
- 보팅과 스택킹 등은 서로 다른 알고리즘 기반이나, 배깅과 부스팅은 대부분 결정 트리 알고리즘 기반
- 앙상블 학습은 수십~수천 개의 분류기 결합하여 다양한 상황 학습하며 결정트리 알고리즘 단점 극복
 - 결정 트리 알고리즘 장점은 취하고 단점은 보완하며 편향-분산 트레이드오프 효과 극대화

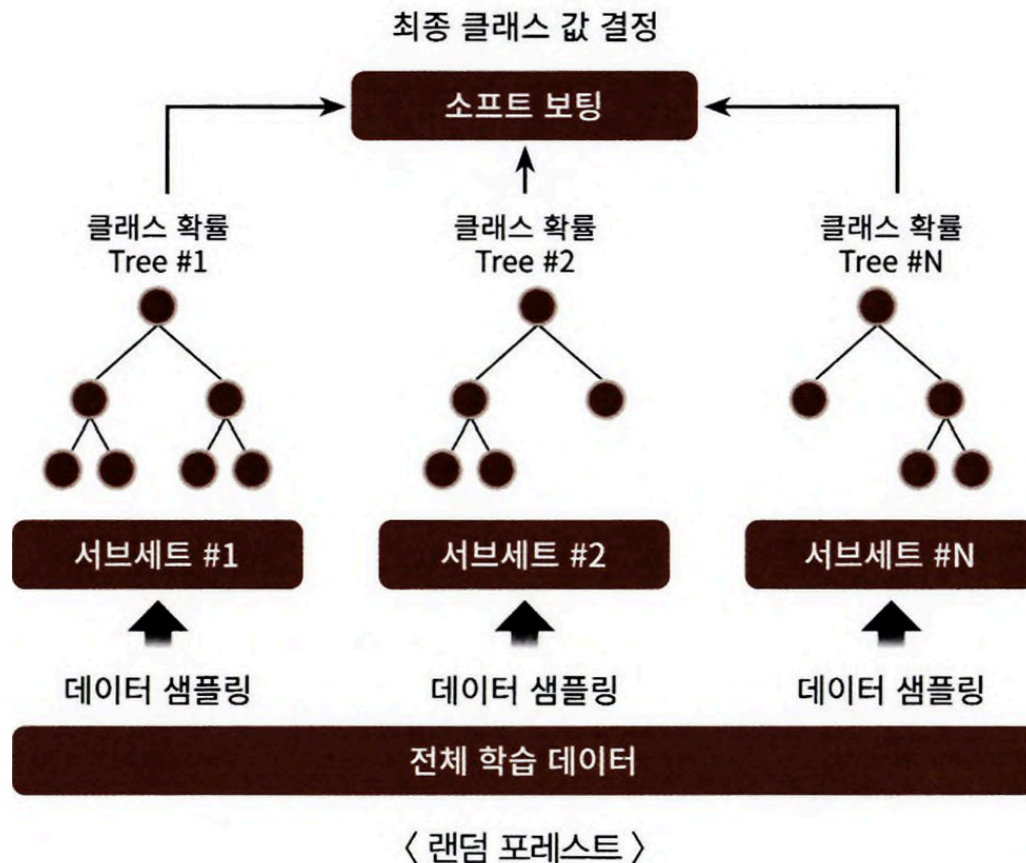
4 랜덤 포레스트

랜덤 포레스트의 개요 및 실습

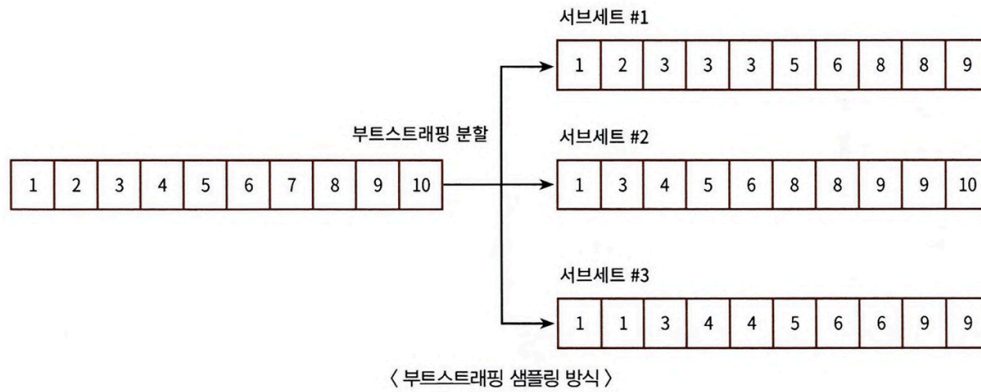
- 배깅은 보팅과 달리 같은 알고리즘으로 여러 개의 분류기 만들어서 보팅으로 최종 결정하는 알고리즘
- 배깅의 대표적 알고리즘 랜덤 포레스트

랜덤 포레스트

- 앙상블 알고리즘 중 비교적 빠른 수행 속도
 - 다양한 영역에서 높은 예측 성능
 - 결정 트리 기반 알고리즘
 - 결정 트리의 쉽고 직관적인 장점 그대로 가짐
1. 여러 개의 결정 트리 분류기가 전체 데이터에서 배깅 방식으로 각자의 데이터를 샘플링해 개별적으로 학습 수행
 2. 최종적으로 모든 분류기가 보팅 통해 예측 결정



- 개별적인 분류기의 기반 알고리즘은 결정 트리
- 개별 트리가 학습하는 데이터 세트는 전체 데이터에서 일부 중첩되게 샘플링한 데이터 세트
 - ⇒ 중첩되게 분리하는 것을 부트스트래핑(bootstrapping)이라 함
 - bagging이 bootstrap aggregating의 준말
 - bootstrap은 원래 통계학에서 여러 개의 작은 데이터 세트를 임의로 만들어 개별 평균의 분포도 측정 목적의 샘플링 방식
- 랜덤포레스트의 서브세트(subset) 데이터는 이러한 부트스트래핑으로 데이터가 임의로 만들어짐
- 서브세트의 데이터 건수는 전체 데이터 건수와 동일하지만
 - 개별 데이터가 중첩되어 만들어짐
- 원본 데이터 건수가 10개인 학습 데이터 세트에 랜덤 포레스트를 3개의 결정 트리 기반으로 학습하려고 $n_estimators=3$ 으로 하이퍼 파라미터 부여 시 서브세트



- 사이킷런은 RandomForestClassifier 클래스 통해 랜덤 포레스트 기반 분류 지원
- 사용자 행동 인식 데이터 세트 예측

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

# 결정 트리에서 사용한 get_human_dataset( )을 이용해 학습/테스트용 DataFrame 반환
X_train, X_test, y_train, y_test = get_human_dataset()

# 랜덤 포레스트 학습 및 별도의 테스트 셋으로 예측 성능 평가
rf_clf = RandomForestClassifier(random_state=0)
rf_clf.fit(X_train, y_train)
pred = rf_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred)
print('랜덤 포레스트 정확도: {0:.4f}'.format(accuracy))
```

랜덤 포레스트 정확도: 0.9223

- random_state를 0으로 설정
 - 재수행마다 동일한 예측 결과 출력 위해
- get_human_dataset() 이용해 DataFrame 가져옴

랜덤 포레스트 하이퍼 파라미터 및 튜닝

트리 기반의 앙상블 알고리즘 단점

- 하이퍼 파라미터 너무 많음

- 그로 인한 튜닝 시간 너무 길
- 시간은 많이 소모되어도 예측 성능이 크게 향상되는 경우 많지 않음
- 그나마 랜덤 포레스트가 하이퍼 파라미터 적은 편
 - 결정 트리에서 사용되는 하이퍼 파라미터와 같은 파라미터가 대부분이기 때문

n_estimators	- 랜덤 포레스트에서 결정 트리의 개수 지정 - 디폴트는 10개 - 많이 설정할수록 무조건 성능 향상 X - 늘릴수록 학습 수행 시간 늘어남 감안 필요
max_features	- 결정 트리의 max_features 파라미터와 같음 - RandomForestClassifier의 기본 max_features는 'None'이 아니라 'auto', 즉 'sqrt'와 같음 ⇒ 랜덤 포레스트의 트리를 분할하는 피처를 참조할 때 전체 피처가 아니라 sqrt(전체 피처 개수)만큼 참조 - e.g.) 전체 피처 16개라면 분할 위해 4개 참조

- max_depth나 min_samples_leaf, min_samples_split와 같이 결정 트리에서 과적합 개선하기 위해 사용되는 파라미터가 랜덤 포레스트에 똑같이 적용

GridSearchCV 이용하여 랜덤 포레스트의 하이퍼 파라미터 튜닝

- GridSearchCV 생성 시 n_jobs=-1 파라미터 추가하면 모든 CPU 코어를 이용해 학습할 수 있음

```
from sklearn.model_selection import GridSearchCV

params = {
    'max_depth' : [8, 16, 24],
    'min_samples_leaf' : [1, 6, 12],
    'min_samples_split' : [2, 8, 16]
}

# RandomForestClassifier 객체 생성 후 GridSearchCV 수행
rf_clf = RandomForestClassifier(n_estimators=100, random_state=0, n_jobs=-1)
grid_cv = GridSearchCV(rf_clf, param_grid=params, cv=2, n_jobs=-1)
grid_cv.fit(X_train, y_train)

print('최적 하이퍼 파라미터:\n', grid_cv.best_params_)
print('최고 예측 정확도: {0:.4f}'.format(grid_cv.best_score_))
```

최적 하이퍼 파라미터:
 {'max_depth': 16, 'min_samples_leaf': 6, 'min_samples_split': 16}
 최고 예측 정확도: 0.9157

다시 RandomForestClassifier 학습 시킨 뒤 예측 성능 측정

```
rf_clf1 = RandomForestClassifier(n_estimators=100, max_depth=16, min_samples_leaf=6, #
                                min_samples_split=2, random_state=0)
rf_clf1.fit(X_train , y_train)
pred = rf_clf1.predict(X_test)
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test , pred)))
```

예측 정확도: 0.9253

feature 중요도 시각화

```
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

ftr_importances_values = rf_clf1.feature_importances_
ftr_importances = pd.Series(ftr_importances_values, index=X_train.columns )
ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]

plt.figure(figsize=(8,6))
plt.title('Feature importances Top 20')
sns.barplot(x=ftr_top20 , y = ftr_top20.index)
fig1 = plt.gcf()
plt.show()
plt.draw()
fig1.savefig('rf_feature_importances_top20.tif', format='tif', dpi=300, bbox_inches='tight')
```

